

From Capability to Assurance in Autonomous Penetration-Testing Harnesses: A Framework and Reference Implementation

Joas Antonio dos Santos
Independent Researcher — AI and Offensive Security
São Paulo, SP, Brazil
joas.santos@redteamleaders.com

Abstract—Research on large-language-model (LLM) agents for penetration testing is evaluated almost entirely by *capability*: whether the agent captures a flag or reproduces a proof-of-concept. This metric is appropriate for a benchmark but silent on the properties that decide whether an autonomous agent can be used in an authorized engagement—whether a reported finding is *true*, whether the agent remained inside its *authorized* scope, and whether an operator can *audit* what it did. We call these *assurance* properties and argue that they belong to the *harness*—the runtime wrapping the model—and can be enforced in code. This paper makes three contributions. First, we define a framework of five assurance properties (evidence grounding, non-destructive claim reduction, computed severity, enforced authorization, and tamper-evident accountability), each with a formal model and an explicit acceptance test, and we connect them to prior work in capability-based security, tamper-evident logging, and software provenance. Second, we position representative systems (PentestGPT, the Cochise reference harness, MAPTA, and the trajectory judge PentestJudge) within the framework using published, coded criteria, and identify a consistent assurance gap. Third, we study one open-source implementation, NeuroSploit (pinned to an exact commit), reporting its architecture, its complexity cost measured by documented commands, and a version-pinned, content-addressed artifact bundle from a run against a public deliberately-vulnerable target. We are precise about what is and is not evidenced: we execute the deterministic P4 and P5 acceptance tests directly and, in doing so, find and report a real enforcement gap, which we reflect by scoring both P4 and P5 as partial. We therefore claim an *initial existence argument* that the properties are realizable together—not a comparative-performance result—and we specify the multi-target, ablation, and adversarial evaluation protocol required to turn the framework’s obligations into measurements. The contribution is a vocabulary, a set of realizable design obligations, and a measurable evaluation plan for autonomous offensive agents whose output can withstand scrutiny.

Index Terms—autonomous penetration testing, large language models, assurance, evidence grounding, capability-based authorization, tamper-evident logging, CVSS, offensive security

I. INTRODUCTION

Autonomous penetration testing has become a measurable engineering discipline. Since Fang et al. showed that an LLM agent could exploit real one-day web vulnerabilities from a CVE description [1], systems such as PentestGPT [2], HPTSA [3], MAPTA [4], CHECKMATE [5], Pentest-R1 [6], and the Cochise reference harness [7] have advanced the state of the art, and benchmarks such as CyberGym [11] and ExploitGym

[12] now measure offensive capability across hundreds to thousands of real targets.

Almost all of this progress is measured with one family of metrics: task or subtask success, operationalized as capturing a flag or reproducing a proof-of-concept (PoC). That instrument is right for a benchmark. It is silent, however, on three properties that determine whether an autonomous agent can be used in a paid, authorized engagement:

- **Truth.** A flag capture is self-verifying; a client report is not. An agent that emits a fluent but unproven vulnerability narrative—the “hallucinated compliance” pattern of Nguyen and Husain [14]—imposes remediation cost and destroys credibility.
- **Authorization.** The engagement boundary belongs to the client. If scope exists only as prompt text, an agent that follows an off-target redirect or acts on a discovered subdomain has crossed a legal line, discovered only by reading the report.
- **Accountability.** Afterwards the operator must state what was done, to what, under whose authority, and with what result. Prose logs are lossy and reorderable.

These are *assurance* questions, and our central claim is that they are properties of the *harness*—the runtime that surrounds the model with a shell, tool wrappers, a token budget, a memory policy, and an output parser—not of the model’s weights. The harness is where an assurance obligation can be enforced in code rather than requested in a prompt, and the same base model in two harnesses can differ by tens of percentage points on the same benchmark [5], [9], which already establishes the harness as the decisive engineering surface. The assurance properties themselves are not new inventions: they instantiate, for offensive agents, long-standing ideas from capability-based security [19]–[21], enforceable security policies [24], tamper-evident logging [25]–[27], and provenance [29], [30]. Our framework’s role is to name which of them a pentest harness owes, and to state each as a testable obligation.

Research questions.

RQ1 What assurance properties should an autonomous penetration-testing harness provide, and can each be stated as an implementation-independent obligation with

an explicit acceptance test?

RQ2 Where do existing systems sit with respect to these properties under objective, coded criteria?

RQ3 Are the properties realizable in a single system, at what complexity cost, and what evaluation would measure how completely each is satisfied?

Contributions.

- 1) A *framework* of five assurance properties (Section IV), each with a formal model and an acceptance test, related to prior security literature; this answers RQ1.
- 2) A *positioning* of representative systems using published coding criteria (Section II and Appendix I), answering RQ2.
- 3) A *reference implementation study* of NeuroSploit [34], pinned to an exact commit, with complexity measured by documented commands and a content-addressed artifact bundle (Sections V–VII); and a *measurable evaluation protocol*—multi-target precision/recall, ablation, and adversarial P4/P5 batteries (Appendix H)—that specifies what remains to be measured. Together these answer RQ3.

We state scope precisely. We make *no* comparative-performance claim: no shared benchmark for assurance properties exists, and constructing one is the central open problem (Section VIII). The reference study is an *initial existence argument*, not a proof that every property holds on every action; Section VII is explicit about what the archived artifacts do and do not evidence.

II. BACKGROUND AND POSITIONING

A. Autonomous offensive agents and their harnesses

Early autonomous LLM pentesters were single-agent ReAct loops [1], [8]. Multi-agent designs followed: HPTSA showed a planner dispatching task-specific subagents outperforms a generalist on zero-day-class web targets [3], and the Planner–Executor–Perceptor pattern was formalized in CHECKMATE, which, holding the base model fixed, obtains a $> 20\%$ success improvement and $> 50\%$ cost reduction from the harness alone [5]. MAPTA added tool-grounded execution with end-to-end exploit validation [4]. Cochise [7] is the closest work in intent: a deliberately small (630 LOC) Python reference harness for reproducible *traces*. The properties we study concern assurance of *output*.

B. Evaluation and judging

Cybench evaluates agents on 40 professional CTF tasks and is reported in most frontier-model system cards [9]. PentestEval shows end-to-end pipeline success collapsing to $\leq 31\%$ even when per-stage scores are high [10]. CyberGym [11] and ExploitGym [12] scale evaluation to real code. PentestJudge [13] argues a flag-only reward is gameable and grades agent *behavior*; that is an assurance argument at the trajectory level, which our framework complements at the level of findings, authorization, and per-action accountability.

TABLE I

ASSURANCE PROPERTIES (P1–P5) ACROSS REPRESENTATIVE SYSTEMS, CODED PER APPENDIX I. ✓=ADDRESSED IN THE ANALYZED ARTIFACT; ~=PARTIAL/PROCESS-LEVEL; —=NOT ADDRESSED IN THE ANALYZED ARTIFACT (ABSENCE OF A CLAIM, NOT PROOF OF ABSENCE IN CODE). NEUROSPLOIT P4/P5 ARE ~ BECAUSE THE PRIMARY-TARGET CHECK (P4) AND EXTERNAL ANCHORING (P5) ARE NOT YET IN THE TESTED BUILD; SEE SECTIONS V-D AND VIII.

	PentestGPT [2]	Cochise [7]	MAPTA [4]	PentestJudge [13]	NeuroSploit [34]
P1 Evidence grounding	—	—	✓	~	✓
P2 Non-destr. reduction	—	—	~	~	~
P3 Computed severity	—	—	—	—	~
P4 Enforced authorization	—	—	—	—	~
P5 Tamper-evident audit	—	—	—	—	~
Trajectory judging	—	~	—	✓	~
Released implementation	✓	✓	✓	✓	✓

C. Security foundations

Each property below has a lineage. P4 is a capability [19], [20]; the concrete token design follows Macaroons [21] and JWT [22] with HMAC [23]. P5 is tamper-evident logging [25], [26] over a hash chain [27], whose full-rewrite weakness is addressed by external anchoring as in Certificate Transparency [28] and supply-chain attestation [29]. P1 relates to grounding and attribution for LLM output [17], [18] and to prompt-injection defenses for tool-using agents [15], [16]. P3 builds on CVSS v3.1 [31], and the enforcement stance follows Schneider’s enforceable security policies [24] and W3C provenance [30].

D. The assurance gap

Appendix I states objective coding criteria and applies them, per analyzed version, to representative systems; Table I summarizes. The pattern is consistent: capability is addressed and, at best, trajectory-level judging; finding-level truth, enforced scope, and per-action accountability are left to the surrounding process. This gap—not any single missing feature—is what the framework names.

III. THREAT MODEL

Setting. An operator runs a harness against systems for which a client has granted written authorization. Operator and authorizing party may differ. The target is partially observable and may be hostile to the agent.

Assets. (A1) client systems and data, including assets adjacent to but outside scope; (A2) report integrity—no false findings, no silently dropped real ones; (A3) the operator’s ability to prove, afterwards, that the engagement stayed within authorization.

Adversaries and failure modes. (T1) the *model* as a source of confident hallucination; (T2) a *hostile target* performing prompt injection through tool output [15], [16]; (T3) *scope drift* onto discovered-but-unauthorized assets, including redirects, DNS rebinding, alternate IP encodings, and mid-run DNS changes; (T4) an *after-the-fact dispute*, including an adversary who can edit, remove, reorder, or wholesale-rewrite the log. The framework addresses T1, T3, and T4 structurally; T2 is reduced but not eliminated (Section VIII).

For T4 we distinguish a *local* adversary (can edit files but not forge external signatures) from a *full-rewrite* adversary (can recompute an unanchored chain); P5’s acceptance test targets both only when external anchoring is present (Appendix G).

IV. A FRAMEWORK OF ASSURANCE PROPERTIES

We state five properties as obligations on the harness, each with a formal model and an *acceptance test*: a concrete check that decides whether an implementation satisfies the property. A *finding* f is a claim about a target; the *evidence ledger* $E = \{e_1, \dots, e_m\}$ is the set of receipts collected during an engagement (raw tool output: an HTTP response, an out-of-band callback, an error oracle, a shell transcript, or, white-box, a source citation). Each receipt e records a target, an action reference, a timestamp, and a content hash.

A. P1 — Evidence grounding

No claim enters the reported world model without a receipt that a decision procedure outside the model accepts. Let $\text{cite}(c) \subseteq E$ be the receipts a claim c names. Define the external decision procedure supports (formalized in Appendix B) returning a verdict in $\{\text{Confirmed}, \text{Rejected}, \text{NeedsReview}\}$. P1 requires

$$\forall c \in f : \text{cite}(c) \neq \emptyset \wedge \text{supports}(c, \text{cite}(c)) \neq \text{Rejected}, \quad (1)$$

with any **NeedsReview** claim labelled and withheld from asserted status. The obligation is that supports is *not* “an LLM agrees”; model agreement is not evidence. *Acceptance test (T-P1)*: inject a syntactically plausible but fabricated receipt (a transcript with no matching action hash); the finding must not reach asserted status.

B. P2 — Non-destructive claim reduction

An overstated finding is reduced to what its evidence supports, never silently deleted or inflated, and a claim with no security-relevant mechanic is not force-retained. Decompose f into separable claims and partition them into supported mechanic $M(f)$ and unsupported impact $I(f)$. The reduction returns one of four typed outcomes (Appendix D): **supported_finding** ($M(f)$ rewritten), **informational_observation** (a benign but true fact), **insufficient_evidence**, or **invalid_claim**. Formally,

$$f' = \begin{cases} \text{rewrite}(M(f)) & M(f) \text{ security-relevant,} \\ \text{demote}(M(f)) & M(f) \text{ true but benign,} \\ \emptyset & \text{otherwise.} \end{cases} \quad (2)$$

Acceptance test (T-P2): a finding whose impact is unproven but whose mechanic is proven (e.g., a measured missing rate limit) must be retained and downgraded, not rejected; a finding with neither must not be force-retained.

C. P3 — Computed, evidence-graded, impact-separated severity

Severity is computed from evidence by a deterministic function and capped by demonstrated impact; potential impact is recorded separately so that safely-unexploited worst cases are not understated. Let g be the CVSS v3.1 base function [31] over a full metric vector whose values each cite a receipt; let $\ell_d(f)$ be the *demonstrated*-impact rung and $\ell_p(f)$ the *potential*-impact rung on

$$\text{reached} < \text{read} < \text{wrote} < \text{RCE} < \text{crossed}. \quad (3)$$

P3 requires a demonstrated score and a separately-recorded potential score,

$$\text{sev}_d(f) = \min(g(f), \text{cap}(\ell_d(f))), \quad \text{sev}_p(f) = g(f), \quad (4)$$

each annotated with an evidence-confidence level. The separation prevents the failure the reviewer of an authorized test rightly fears: correctly declining to demonstrate destructive impact must not silently lower the recorded risk. Appendix E gives the cap table and a worked example. *Acceptance test (T-P3)*: two findings of the same CWE with different demonstrated impact must receive different sev_d ; class-only scoring fails.

D. P4 — Enforced, attributable authorization

Scope is a signed grant, bound to a client, an operator, an engagement, and an asset set, that in-session behavior can narrow but never widen, with replay and revocation controls. Let A be an authorization token carrying issuer (client key id), audience (harness instance), engagement id, operator id, scope (hosts, CIDRs, URL prefixes), action ceiling, environment, validity window $[nbf, exp]$, and nonce; let σ be its signature. P4 requires that an action a on target t be admitted only if

$$\begin{aligned} & \text{verify}(A, \sigma, K_{\text{iss}}) \wedge nbf \leq \text{now} \leq exp \wedge \text{fresh}(\text{nonce}) \\ & \wedge \text{aud}(A) = \text{self} \wedge (t, a) \sqsubseteq A \wedge (t, a) \sqsubseteq \text{cfg}, \end{aligned} \quad (5)$$

where cfg is local configuration and $\sqsubseteq$ is “within.” A is a ceiling: no in-session reasoning can yield $(t, a) \not\sqsubseteq A$. Appendix F specifies the token, key custody, revocation, and rotation, and notes that HMAC [23] proves only holder-of-key, so separating client from operator argues for an asymmetric signature (client-held private key, harness-held public key), which we present as the recommended evolution of the current HMAC form. *Acceptance test (T-P4)*: the adversarial scope battery of Appendix H (off-scope redirect, DNS rebinding, alternate IP encodings, userinfo URLs, over-broad wildcard, expired/tampered token, in-session widening) must each be denied.

E. P5 — Externally-anchored tamper-evident accountability

Every action is recorded in an append-only log whose integrity is verifiable, and whose root is anchored outside the writing process so that a full rewrite is detectable. Actions produce records $r_1, \dots, r_n$ chained by $h_i = H(r_i \parallel h_{i-1})$, $h_0 = H(\text{genesis})$. A hash chain alone detects only local

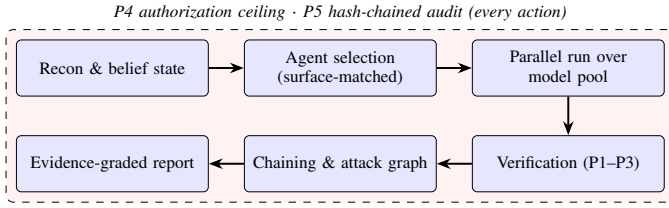


Fig. 1. NeuroSploit pipeline. Recon/select/run/verify/chain/report are wrapped by the P4 ceiling and the P5 chain.

edits; a full-rewrite adversary can recompute it. P5 therefore additionally requires periodic anchoring of h_i —an external signature, a trusted timestamp, a write-once store, or publication to a transparency log [26], [28]—so that no post-hoc chain can match the anchored roots. *Acceptance test (T-P5)*: the tamper battery of Appendix H (record removed, altered, reordered; evidence substituted; token swapped; log truncated; whole-chain rebuilt) must each be detected; the last only when anchoring is enabled.

Relationships and completeness. P1 is prerequisite to P2 and P3. P4 and P5 are independent of P1–P3 and of the model. The five are proposed as necessary for an engagement that must withstand scrutiny; they are not claimed exhaustive—trajectory-level judging [13] is a candidate sixth, kept complementary because it ranges over runs rather than findings or actions.

V. REFERENCE IMPLEMENTATION

To address RQ3 we study **NeuroSploit** [34], an open-source Rust harness developed by the author (disclosed as a competing interest, Section IX). It is used as an existence argument and a source of complexity measurements, not a product claim; any harness satisfying P1–P5 would serve. All measurements in this paper are pinned to one commit (Table II) and produced by the commands in Appendix A.

TABLE II
EVALUATED VERSION (ALL SECTION VI–VII NUMBERS REFER TO THIS STATE).

Item	Value
Commit SHA	8894649ccb42a04c4763085b3b663e49fd98ddf5
Describe/tag	v4.0.0-32-g8894649
Commit date	2026-09-18
Measurement date	2026-09-18
OS	macOS 15.1 (Darwin 24.1.0, arm64)
Toolchain	rustc/cargo 1.96.0

Figure 1 shows the pipeline: recon builds a belief state; a selector chooses surface-matched agents from a markdown knowledge base; agents run in parallel over a model pool; outputs pass the verification pipeline; survivors are chained and reported. The P4 authorization ceiling and the P5 audit chain wrap every action.

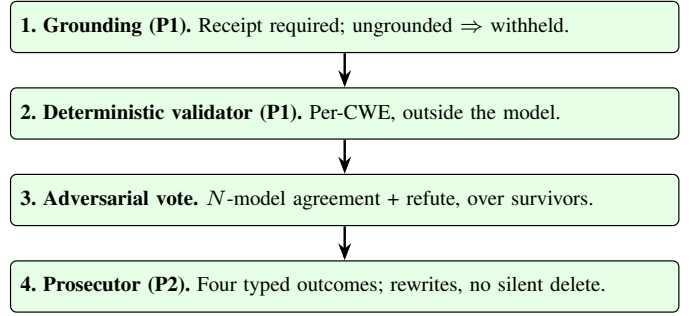


Fig. 2. Verification pipeline realizing P1–P2; severity (P3) follows.

A. Planning as an anti-hallucination gate

The target is partially observable, so the harness tracks a *belief*: a property graph whose nodes carry a probability and an evidence count, updated from observations by a Bayesian step; per-node Shannon entropy $H(b)$ measures diffuseness. The “recon vs. exploit” choice is a value-of-information decision: with $V_{\text{obs}}(b)$ and $V_{\text{exploit}}(b)$ the expected payoffs, the planner observes while

$$V_{\text{obs}}(b) > V_{\text{exploit}}(b), \quad (6)$$

which holds when the belief is diffuse; the same inequality bars asserting exploitability while $H(b)$ is high, reinforcing P1 at the planning layer.

B. Realizing P1–P3

Grounding (P1) enforces a receipt in the engagement-appropriate mode (empirical / symbolic / either). Deterministic per-CWE validators (Appendix C) implement supports outside the model: 27 validators, keyed to CWE classes and referencing 31 CWE identifiers, each returning *Confirmed/Rejected/NeedsReview*. Adversarial N -model voting runs only over survivors. The evidence prosecutor realizes P2 with the four typed outcomes above. Severity (P3) is then computed and impact-separated (Appendix E). Figure 2 shows the order; deterministic checks precede model-based ones.

C. Realizing P4–P5

Authorization is a signed capability token (Appendix F) verified as the P4 ceiling, with a two-layer scope guard: hard scope (allowlist defaulting to the engagement target, so discovery cannot widen scope) and soft scope (read-only zones, forbidden methods, rate ceilings). Every action emits a hash-chained record (Appendix G) with a `verify()` routine; external anchoring is discussed there as the defense against full rewrite. An additive OT risk model $r_{\text{eff}} = r_{\text{action}} + c_{\text{asset}} + r_{\text{proto}} + p_{\text{priv}} + b_{\text{radius}}$ forbids disruptive industrial function codes while permitting the reads OT findings come from. These mechanisms are exercised both by the unit-test suite (Section VI) and by a command-line acceptance battery we ran directly (Section V-D); the latter also surfaced a real enforcement gap, reported below.

D. Executed acceptance tests for P4 and P5

Because the P4 and P5 acceptance tests are deterministic and need no model, we executed them on the pinned build (Table II). Table III reports the P4 capability-token battery. Token issuance, verification, tamper rejection, wrong-key rejection, and expiry detection all behave as specified, and `constrain()` correctly drops an out-of-grant host from local scope (the audit record for an off-grant run shows the hard scope reduced to the granted host). *However*, when the run’s *primary target* lies outside the grant, the offline engagement still reconned it and logged `engagement-start` as `allow`: the token constrains the agent scope but the tested build does not refuse an out-of-grant primary target at engagement start. We therefore score P4 as *partial* (Table I) and list this as a limitation (Section VIII). For P5, an offline run produced a hash-chained `audit.jsonl` (fields `seq`, `prev_hash`, `hash= sha256(digest)`), and the audit module’s tests—the `chain_verifies_and_notifies_tampering`, `removing_a_record_breaks_the_chain_too`, `reopening_continues_the_same_chain`, `refusals_are_recorded_not_just_actions`—pass, covering the alter/remove/reorder cases of T-P5; external anchoring against a full-rewrite adversary remains design-only. Of the harness’s 312 passing unit tests, 109 exercise the P1–P5 modules directly—P1 (grounding, validation) 52, P2 (claims, prosecutor) 19, P4 (capability, scope) 27, P5 (audit) 11—while the remainder cover recon, planning, chaining, and reporting; P3’s severity-cap logic is covered within the P1 validation and P2 claim tests rather than a dedicated module. The full suite reports 312 passed at the pinned commit.

TABLE III
EXECUTED P4 CAPABILITY-TOKEN BATTERY (CLI, DETERMINISTIC).

Case	Expected	Observed
Valid token	verify	verified, grant printed
Tampered token	refuse	refused (signature)
Wrong signing key	refuse	refused
Expired (<i>exp</i> past)	refuse	flagged expired
Off-grant host in local scope	drop	dropped from hard scope
Off-grant <i>primary target</i>	refuse	reconned (gap)

VI. IMPLEMENTATION COST

Table IV reports the pinned instance, measured by Appendix A. Enforcing P1–P5 is not free: the validators, token and audit machinery, and multi-model verification account for a substantial fraction of the code, and the multi-model vote adds per-finding token and latency cost over a single-model pipeline (quantified by the protocol of Appendix H). The scale itself is a data point for RQ3: assurance is realizable at tens of modules and tens of thousands of lines, an order of magnitude beyond a trace-reproduction harness such as Cochise (630 LOC [7]), which is consistent with the gap being real engineering rather than a prompt.

TABLE IV
COMPLEXITY OF THE PINNED INSTANCE (COMMANDS IN APPENDIX A).

Component	Metric
Rust modules (harness crate)	40
Rust LOC (harness crate)	23,370
Rust LOC (workspace, excl. target)	28,802
Automated tests (<code>#[test]/#[tokio::test]</code>)	312
Deterministic validators (impl <code>CweValidator</code>)	27
CWE identifiers referenced by validators	31
Agent definitions (markdown)	446
Supported model providers	18

VII. REFERENCE RUN AND ARTIFACT BUNDLE

To characterize behavior—not to claim a comparative result—we examine an archived black-box run against `http://testaspnet.vulnweb.com/`, a public deliberately-vulnerable application published by Acunetix for testing (an ASP.NET/IIS 8.5 site; recon recorded zero security headers and a non-Secure ASP.NET_SessionId cookie). The run scheduled 245 surface-matched agents and produced 27 findings after verification, from 25 distinct agents, spanning 13 CWE classes, in 3.55 hours, with 32 PoC scripts and 27 evidence screenshots (Table V).

TABLE V
ARCHIVED REFERENCE RUN (TESTASPNET.VULNWEB.COM, BLACK-BOX).

Quantity	Value
Agents scheduled	245
Agents producing a verified finding	25
Findings after verification	27
Critical / High / Medium / Low / Info	1 / 6 / 14 / 4 / 2
Distinct CWE classes	13
PoC scripts / evidence screenshots	32 / 27
Wall-clock time	3.55 h
Artifact bundle root (SHA-256)	5e018a1e...55bce3

What the archived run evidences, precisely. The artifacts record, per finding, the receipt text, PoC reference, adversarial votes, a validated flag, a `review_status`, and a severity *label* (P1, P2, and the labelling half of P3). They do *not* contain populated CVSS *vectors* (the `cvss` field is empty for all 27), nor a capability token, nor an audit-log file: this run predates full instrumentation of those outputs. CVSS-vector computation (P3) is evidenced by the code and its 312 unit tests, not by this archived run. Token verification (P4) and the hash-chained audit log (P5) are evidenced separately by the executed acceptance battery of Section V-D, which produced a real capability token and a hash-chained `audit.jsonl` and also revealed the P4 primary-target gap. The archived target run itself contains no audit log; the bundle additionally contains a separate `audit.jsonl` produced by the deterministic P5 acceptance run (Section V-D) and must not be read as the audit trail of the archived engagement. The archived run’s model version, temperature, vote count,

token budget, concurrency, and seed were not persisted and are reported as *unknown*; the harness default at that time was `anthropic:claude-opus-4-8`. Producing a single fully-instrumented *engagement* run that carries all five properties’ artifacts end-to-end—with those parameters recorded—is part of the protocol in Appendix H.

Reproducibility. The non-sensitive artifacts (recon, findings, PoCs, exploitation notes, the executed acceptance results, and the produced `audit.jsonl`; the public target contains no third-party data) form a *content-addressed* bundle `assurance-artifacts-v1.tar.gz` (40 files, 84 KB, MIT-licensed), available from the project repository [34] at the pinned commit (Table II), with per-file SHA-256 manifest root

```
5e018a1e1a9dc2d19dd82eb0fb3e8af2
7351357f72932d7e7a7df5ae8655bce3
```

The published hash is a content identifier for the bundle, *not* an external anchor for the audit chain (Appendix G). Appendix A lists the commands to recompute every number and the manifest root, and the pinned commit (Table II) fixes the code.

An honest negative. On separate runs against `testphp.vulnweb.com` (a PHP/MySQL sibling known to contain SQL injection), the instance scheduled 245 agents but reported *zero* confirmed findings. We report this. The deterministic SQLi validator (CWE-89) requires a reproducible baseline-versus-attack differential; those runs terminated at the observation phase—consistent with Equation (6)—before an agent produced one. The behavior is correct by P1 yet is a false *negative* against a truly vulnerable target, illustrating that a strict evidence rule trades recall for precision. The protocol of Appendix H is designed to quantify exactly this trade-off.

VIII. DISCUSSION, LIMITATIONS, AND THREATS TO VALIDITY

No assurance benchmark; single instrumented run (primary threats). No shared, public benchmark measures truth-under-scrutiny, scope enforcement, or auditability, so every ranking claim is out of scope, and the empirical evidence here is one archived behavioral run plus a unit-tested codebase. The paper’s claim is an *initial existence argument*, not a demonstration that each property holds on every action. Appendix H specifies the missing measurements: at least three to five vulnerable applications with known ground truth (e.g., OWASP Juice Shop, WebGoat, DVWA at fixed versions, a deliberately-vulnerable API, one authorized real scope), ≥ 3 runs per target, and reported true/false positives and negatives, precision, recall, cost, tokens, and time; an ablation removing each of P1’s deterministic validators, the vote, the prosecutor, and the P3 cap, and comparing common logging against P5; adversarial P4 and P5 batteries; and baselines (NeuroSploit with P1–P5 disabled, a plain ReAct loop, a single-model configuration, and a conventional scanner on the same targets).

Possible memorization. The archived run targets a widely-used public application that may appear in training data, which

could inflate recall; the honest negative partially counters an over-optimistic reading, but external validity is limited until diverse, partly-private targets are used.

Precision–recall. P1’s outside-the-model rule maximizes precision and can suppress true positives (the `testphp` negative); whether the resulting report is more useful than a higher-recall one is the empirical question the protocol answers.

Residual surface (T2). No harness eliminates prompt injection through hostile tool output; adversarial content can influence an agent’s next step before verification runs. P1 limits the *consequences*—an injected instruction cannot manufacture a receipt that passes a per-CWE validator—but the planning layer remains a target, and confining every agent-authored command is only partial in the current implementation.

P4 primary-target gap (found by executing T-P4). The acceptance battery of Section V-D revealed that, while the token correctly constrains agent scope, an offline engagement whose *primary target* lies outside the grant is reconned and logged `allow` rather than refused. This is a concrete enforcement gap between the specification of P4 and the tested build; it is the reason P4 is scored partial, and closing it (checking the primary target against the constrained scope at engagement start) is the first fix the reference implementation owes.

Property strength. As specified, P4’s HMAC form proves holder-of-key, not client intent; the asymmetric evolution (Appendix F) is required to bind client and operator cryptographically. P5’s guarantee against a full-rewrite adversary holds only with external anchoring enabled (Appendix G); without it the mechanism detects local, partial tampering only. We state these as scoped guarantees, not absolutes.

IX. ETHICS AND RESPONSIBLE USE

All runs discussed were conducted against targets published by their owners for security testing (the Acunetix `vulnweb.com` family); no third-party data, credentials, or personal information are disclosed, and evidence contains only synthetic markers chosen by the harness. Autonomous offensive tooling is dual-use; the assurance properties defined here—enforced authorization (P4), tamper-evident accountability (P5), and the OT safety model—are precisely the mechanisms that constrain misuse and make an authorized engagement defensible. We describe an architecture and release no exploit payloads for unpatched third-party vulnerabilities.

Competing interests and funding. The author is the developer of NeuroSploit, the reference instance; this is the paper’s sole competing interest and the reason the study is framed as an existence argument rather than a comparison. The work received no external funding.

Availability. The implementation is open source [34], pinned by Table II; the run artifact bundle is anchored by the manifest root in Section VII, enabling independent inspection.

X. CONCLUSION

The useful frontier for autonomous penetration testing is shifting from capability to *assurance*, and assurance is a

property of the harness. We defined five implementation-independent properties, each with a formal model and an acceptance test, related them to capability-based security, tamper-evident logging, and provenance, positioned existing systems against them, and studied one open-source implementation as a version-pinned, content-addressed existence argument—while stating precisely what its archived run does and does not evidence and specifying the multi-target, ablation, and adversarial protocol that would turn the framework’s obligations into measurements. Building that assurance benchmark is the work we hope this framework enables.

ACKNOWLEDGMENT

The author thanks the maintainers of the public vulnerable-target and benchmark ecosystems. Generative language-model tooling assisted with drafting, reference verification, and copy-editing; all sources were checked by the author against their original records, and the author takes full responsibility for the content, including any errors.

APPENDIX A REPRODUCIBILITY COMMANDS

All numbers in Tables II–V were produced at the pinned commit by the following, on the environment of Table II.

```
# version
git rev-parse HEAD
git describe --tags --always
git log -l --format=%ci
rustc --version

# complexity (Table III)
find agents_md -name '*.md' -type f | wc -l # agents: 446
find crates/harness/src -name '*.rs' | wc -l # modules: 40
find crates/harness/src -name '*.rs' -print0 \
  | xargs -0 cat | wc -l # LOC: 23370
grep -rE '#\[test\]\|#[tokio::test]\| \
  crates/harness/src | wc -l # tests: 312
grep -rE 'impl CweValidator for' \
  crates/harness/src/validation.rs | wc -l # validators: 27

# build + tests
cargo test --locked

# a fresh, fully-instrumented run (protocol, App. F)
neurosploit run <target> \
--capability-token <path> \
--model <provider:model@version> \
--votes <n> --seed <seed> \
--config artifact/config.yaml \
--audit artifact/audit.log
# integrity
shasum -a 256 <each artifact> > MANIFEST.sha256
shasum -a 256 MANIFEST.sha256 # bundle root
```

APPENDIX B THE SUPPORTS DECISION PROCEDURE

Input: a claim c , its cited receipts $\text{cite}(c)$, the engagement mode, and the target/action binding of each receipt. **Output:** Confirmed, Rejected, or NeedsReview. **Procedure.** (1) *Binding:* each receipt’s action hash must match a logged action against an in-scope target and postdate that action; a receipt that cannot be bound is dropped. (2) *Class dispatch:* route to the per-CWE validator (Appendix C); a class with no validator yields NeedsReview (never auto-Confirmed). (3) *Composition:* all receipt *roles* the class requires must be

present (e.g., IDOR requires an owner-resource receipt *and* a cross-identity receipt). (4) *Contradiction:* if any receipt refutes the claim (e.g., the anonymous request was denied), return Rejected. (5) *Verdict:* Confirmed iff roles present, bound, and the class predicate holds; else NeedsReview. The procedure is deterministic and model-free; it is the operational meaning of “outside the model” in P1.

APPENDIX C DETERMINISTIC VALIDATOR CATALOG

The 27 validators reference 31 CWE identifiers, including CWE-16, 22, 78, 79, 89, 113, 200, 209, 306, 307, 319, 347, 352, 384, 524, 525, 530, 548, 601, 611, 614, 639, 644, 650, 863, 915, 918, 942, 1004, 1021, 1336. Representative predicates: **SQLi (89)** a reproducible baseline-vs-attack differential; **XSS (79)** a browser executing a harness-chosen marker (proof by execution); **IDOR (639)** identity B reading identity A’s resource with content-overlap confirmation, rejecting identical identities or generic login-page bodies; **SSRF (918)** an out-of-band callback carrying the marker or retrieval of a controlled internal resource; **missing authentication (306)** a protected resource served to an anonymous request with content match, rejecting the case where the anonymous request is denied.

APPENDIX D THE EVIDENCE PROSECUTOR

The prosecutor answers, in order: what was observed; which sentence exceeds it; what would make the impact factual; and whether the finding survives removing the impact sentence. It returns one of {supported_finding, informational_observation, insufficient_evidence, invalid_claim}; it cannot silently delete, and it cannot force-retain a claim with no security-relevant mechanic (which becomes informational_observation or insufficient_evidence). The finding is rebuilt from the returned minimal supported claim, decoupling the reduction (this component) from rejection (the vote and validators), which is what prevents a proven mechanic from being lost to an overstated headline.

APPENDIX E SEVERITY CAP AND WORKED EXAMPLE

Severity uses the CVSS v3.1 base metrics [31]; each metric value must cite a receipt. The demonstrated-impact ladder (3) caps the base score:

Rung	Typical impact metrics	cap (base)
reached	C:N/I:N/A:N, no data	≤ 6.9 (Med)
read	C:L/H, I:N	≤ 8.9 (High)
wrote	I:L/H	≤ 9.4
RCE	C:H/I:H/A:H	≤ 10.0 (Crit)
crossed	scope S:C, C/I/A:H	≤ 10.0 (Crit)

Example. A reflected SQL error on login.aspx with no data extraction: vector AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:N yields a low base, and $\ell_d = \text{reached}$ caps at Med; the *potential* score sev_p records C:H/I:H (a full extraction) so the undemonstrated worst case is not lost. A NeedsReview metric

leaves the finding at label-only severity with no computed base until resolved.

APPENDIX F CAPABILITY TOKEN AND KEY CUSTODY

Wire form: `ns-cap.v1.<b64url(payload)>.<b64url(mac)>`. Payload fields: `iss` (client key id), `aud` (harness instance), `eng` (engagement id), `op` (operator id), `scope` (hosts/CIDRs/URL prefixes), `ceiling` (max action kind, risk), `env`, `nbfs`, `exp`, `nonce`. Verification follows Section IV. *Custody*: the current HMAC form uses a shared secret and proves holder-of-key only; to bind client and operator, an asymmetric signature (client-held Ed25519 private key, harness-held public key) is the recommended evolution, following the caveat model of Macaroons [21] and the registered-claims model of JWT [22]. *Replay*: `nonce` freshness plus short `exp`. *Revocation*: a revocation list keyed by `eng/iss` and short lifetimes. *Rotation*: `iss` key id selects the verifying key, enabling overlap during rotation.

APPENDIX G AUDIT LOG SCHEMA AND ANCHORING

Each record carries timestamp, agent, hypothesis, action, target, policy_decision, tool, result, evidence_hash, capability_token, and $h_i = H(r_i || h_{i-1})$. `verify()` recomputes the chain and localizes the first break. Against a local adversary this detects removal, alteration, and reordering. Against a full-rewrite adversary, external anchoring is required: periodic signing of h_i with a key outside the run, a trusted timestamp, a write-once store, or publication of roots to a transparency log [25], [26], [28]. The acceptance test T-P5 exercises removal, alteration, reordering, evidence substitution, token swap, truncation, and whole-chain rebuild; the last passes only with anchoring enabled.

APPENDIX H EVALUATION PROTOCOL (TO BE EXECUTED)

This protocol specifies the measurements not yet performed; it is stated so results are comparable when produced. The deterministic P4 and P5 batteries below were partially executed already (Section V-D, Table III); the multi-target, ablation, and baseline measurements, which require live model runs, remain to be executed. **Targets**: ≥ 3 –5 applications with published ground truth—OWASP Juice Shop, WebGoat, DVWA (pinned versions), a deliberately-vulnerable API, and one authorized real scope—each with an enumerated expected-vulnerability list. **Design**: ≥ 3 runs per target with fixed model/version, temperature, vote count, seed, token budget, and concurrency, all recorded in the run config. **Metrics**: true positives, false positives, false negatives against ground truth; precision, recall; cost, tokens, wall-clock; and, for P3, the CVSS delta between evidence-graded and by-class scoring. **Ablation**: remove each deterministic validator, the vote, the prosecutor, and the P3 cap; run with and without P4; compare

common logging vs. anchored P5—reporting the change in false-positive rate and cost attributable to each. **Baselines**: NeuroSploit with P1–P5 disabled, a plain ReAct loop, a single-model configuration, and a conventional scanner on the same targets. **Adversarial P4 (T-P4)**: off-scope redirect, DNS re-binding, alternate IPv6/decimal/hex encodings, CNAME to external host, userinfo URL, over-broad wildcard, unauthorized port, mid-run DNS change, discovered subdomain, expired token, tampered token, in-session widening—each must be denied and logged. **Adversarial P5 (T-P5)**: record removed, altered, reordered; evidence substituted; token swapped; log truncated; whole chain rebuilt—each must be detected (the last with anchoring).

APPENDIX I COMPARATIVE CODING PROTOCOL (TABLE I)

Each cell of Table I was assigned from the analyzed artifact of each system, using fixed criteria. **Analyzed artifacts**: PentestGPT [2] (paper and public repository), Cochise [7] (paper and 630-LOC release), MAPTA [4] (paper), Pentest-Judge [13] (paper), and NeuroSploit [34] (source at the pinned commit, Table II). **Coding rule**: $\checkmark$ = the artifact describes or implements a mechanism meeting the property’s acceptance test (Section IV); $\sim$ = a partial or process-level mechanism that does not fully meet the test; $—$ = the artifact makes no claim to the property. An $—$ records the absence of a claim in the analyzed artifact and is *not* evidence of absence in code that was not reviewed. **Per-property predicate**: P1—an out-of-model check that a receipt supports a claim; P2—a documented rule that reduces rather than deletes an overstated finding; P3—a computed, evidence-graded severity (not by-class); P4—a signed, verifiable scope grant enforced as a ceiling; P5—an append-only, integrity-verifiable action log. **NeuroSploit assignments**: P1 $\checkmark$ (grounding and 27 validators, Section V-D); P2 $\checkmark$ (prosecutor with four typed outcomes); P3 $\sim$ (calculator and cap implemented, but CVSS vectors not populated in the archived run); P4 $\sim$ (token integrity enforced, primary-target check missing); P5 $\sim$ (hash chain present and tested, external anchoring design-only). A second independent coder is recommended before peer review; the criteria are published here so the coding is contestable.

REFERENCES

- [1] R. Fang, R. Bindu, A. Gupta, and D. Kang, “LLM Agents can Autonomously Exploit One-day Vulnerabilities,” arXiv:2404.08144, 2024.
- [2] G. Deng et al., “PentestGPT: Evaluating and Harnessing Large Language Models for Automated Penetration Testing,” in *Proc. 33rd USENIX Security Symp.*, 2024. arXiv:2308.06782.
- [3] Y. Zhu, A. Kellermann, A. Gupta, P. Li, R. Fang, R. Bindu, and D. Kang, “Teams of LLM Agents can Exploit Zero-Day Vulnerabilities,” arXiv:2406.01637, 2024.
- [4] I. David and A. Gervais, “Multi-Agent Penetration Testing AI for the Web (MAPTA),” arXiv:2508.20816, 2025.
- [5] L. Wang et al., “Automated Penetration Testing with LLM Agents and Classical Planning (CHECKMATE),” arXiv:2512.11143, 2025.
- [6] H. Kong, D. Hu, J. Ge, L. Li, H. Li, and T. Li, “Pentest-R1: Towards Autonomous Penetration Testing Reasoning Optimized via Two-Stage Reinforcement Learning,” arXiv:2508.07382, 2025.
- [7] A. Happe and J. Cito, “Cochise: A Reference Harness for Autonomous Penetration Testing,” arXiv:2605.11671, 2026.

- [8] R. Fang, R. Bindu, A. Gupta, Q. Zhan, and D. Kang, "LLM Agents can Autonomously Hack Websites," arXiv:2402.06664, 2024.
- [9] A. K. Zhang et al., "Cybench: A Framework for Evaluating Cybersecurity Capabilities and Risks of Language Models," arXiv:2408.08926, 2024.
- [10] R. Yang, M. Cheng, G. Deng, T. Zhang, J. Wang, and X. Xie, "PentestEval: Benchmarking LLM-based Penetration Testing with Modular and Stage-Level Design," arXiv:2512.14233, 2025.
- [11] Z. Wang, T. Shi, J. He, M. Cai, J. Zhang, and D. Song, "CyberGym: Evaluating AI Agents' Real-World Cybersecurity Capabilities at Scale," arXiv:2506.02548, 2025.
- [12] Z. Wang et al., "ExploitGym: Can AI Agents Turn Security Vulnerabilities into Real Attacks?" arXiv:2605.11086, 2026.
- [13] S. Caldwell, M. Harley, M. Kouremetis, V. Abruzzo, and W. Pearce, "PentestJudge: Judging Agent Behavior Against Operational Requirements," arXiv:2508.02921, 2025.
- [14] V. K. Nguyen and M. I. Husain, "Penetration Testing of Agentic AI: A Comparative Security Analysis Across Models and Frameworks," arXiv:2512.14860, 2025.
- [15] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023.
- [16] Y. Yang, C. Gao, D. Wu, Y. Chen, Y. Li, and S. Wang, "MCPSecBench: A Systematic Security Benchmark and Playground for Testing Model Context Protocols," arXiv:2508.13220, 2025.
- [17] L. Gao et al., "RARR: Researching and Revising What Language Models Say, Using Language Models," in *Proc. ACL*, 2023.
- [18] P. Manakul, A. Liusie, and M. J. F. Gales, "SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models," in *Proc. EMNLP*, 2023.
- [19] J. B. Dennis and E. C. Van Horn, "Programming Semantics for Multi-programmed Computations," *Comm. ACM*, vol. 9, no. 3, pp. 143–155, 1966.
- [20] M. S. Miller, K.-P. Yee, and J. Shapiro, "Capability Myths Demolished," Systems Research Lab, Johns Hopkins Univ., Tech. Rep. SRL2003-02, 2003.
- [21] A. Birgisson, J. G. Politz, Ú. Erlingsson, A. Taly, M. Vrabie, and M. Lenczner, "Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud," in *Proc. NDSS*, 2014.
- [22] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, 2015.
- [23] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-Hashing for Message Authentication," RFC 2104, IETF, 1997.
- [24] F. B. Schneider, "Enforceable Security Policies," *ACM Trans. Inf. Syst. Secur.*, vol. 3, no. 1, pp. 30–50, 2000.
- [25] B. Schneier and J. Kelsey, "Secure Audit Logs to Support Computer Forensics," *ACM Trans. Inf. Syst. Secur.*, vol. 2, no. 2, pp. 159–176, 1999.
- [26] S. A. Crosby and D. S. Wallach, "Efficient Data Structures for Tamper-Evident Logging," in *Proc. 18th USENIX Security Symp.*, 2009.
- [27] R. C. Merkle, "A Digital Signature Based on a Conventional Encryption Function," in *Advances in Cryptology (CRYPTO)*, 1987.
- [28] B. Laurie, A. Langley, and E. Kasper, "Certificate Transparency," RFC 6962, IETF, 2013.
- [29] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappos, "in-toto: Providing Farm-to-Table Guarantees for Bits and Bytes," in *Proc. 28th USENIX Security Symp.*, 2019.
- [30] L. Moreau and P. Missier (eds.), "PROV-DM: The PROV Data Model," W3C Recommendation, 2013.
- [31] FIRST.org, "Common Vulnerability Scoring System v3.1: Specification Document," 2019. <https://www.first.org/cvss/v3.1/specification-document>. Accessed: 2026-09-18.
- [32] OWASP Foundation, "OWASP Top 10:2021," 2021. <https://owasp.org/Top10/>. Accessed: 2026-09-18.
- [33] MITRE Corporation, "MITRE ATT&CK Framework," 2023. <https://attack.mitre.org/>. Accessed: 2026-09-18.
- [34] J. A. dos Santos, "NeuroSploit: An Autonomous Multi-Model Penetration-Testing Harness," open-source software, commit 8894649, 2026. <https://github.com/JoasASantos/NeuroSploit>. Accessed: 2026-09-18.